



**PUC Minas**

Arquitetura de Computadores III

# Modelo de Desempenho e Benchmarks para SMP

- Hoje vamos discutir como avaliar o desempenho de sistemas multiprocessados.
- O foco está em dois temas principais: **benchmarks paralelos** e **modelos de desempenho**.
- **Benchmarks** permitem comparar sistemas reais, mas exigem regras rígidas para evitar resultados artificiais.
- **Modelos de desempenho** ajudam a explicar por que uma aplicação é limitada por computação ou por memória.
  - O **Roofline Model** será usado como principal exemplo de modelo para arquiteturas paralelas.

# Modelo de Desempenho e Benchmarks para SMP

## Por que benchmarks são sensíveis?

- Benchmarks influenciam diretamente a reputação de arquiteturas, compiladores e sistemas comerciais.
- Um sistema “vencedor” pode ganhar visibilidade técnica, comercial e acadêmica.
- Por isso, os resultados precisam refletir melhorias reais, e não apenas otimizações artificiais para um teste específico.
- **Benchmark confiável** é aquele que mede desempenho de forma comparável, reproduzível e relevante.
- Regras rígidas são usadas para impedir que o resultado seja manipulado por truques de implementação.

# Modelo de Desempenho e Benchmarks para SMP

## Regras tradicionais de benchmarks

- Em benchmarks clássicos, normalmente não se pode alterar o código-fonte, os dados de entrada ou o resultado esperado.
- O programa deve produzir uma única resposta correta, permitindo comparação objetiva entre sistemas.
- Qualquer desvio das regras invalida o resultado, pois compromete a justiça da comparação.
- Essas restrições isolam melhor o efeito da arquitetura e do compilador.
- Porém, elas também limitam inovações em algoritmos, estruturas de dados e linguagens.

# Modelo de Desempenho e Benchmarks para SMP

## Strong scaling e Weak scaling

- **Strong scaling** avalia como o tempo de execução muda ao aumentar o número de processadores mantendo o problema fixo.
- **Weak scaling** permite aumentar o tamanho do problema conforme o número de processadores cresce.
- Muitos benchmarks multiprocessados aceitam weak scaling porque sistemas paralelos podem variar muito em escala.
- Weak scaling, a comparação deve ser feita com cuidado, pois sistemas podem estar resolvendo problemas de tamanhos diferentes.
- Strong scaling é mais rígido, mas pode ser injusto para arquiteturas projetadas para problemas muito grandes.

# Modelo de Desempenho e Benchmarks para SMP

## Linpack

- Linpack é uma coleção de rotinas de álgebra linear, com foco no benchmark de eliminação Gaussiana.
- A rotina DGEMM representa apenas uma parte do código, mas domina grande parte do tempo de execução.
- O benchmark permite **weak scaling**, deixando o usuário escolher o tamanho do problema.
- Supercomputadores podem resolver matrizes densas de dimensões extremamente grandes.
- Linpack mede principalmente desempenho em operações de ponto flutuante densas e altamente otimizáveis.

# Modelo de Desempenho e Benchmarks para SMP

## Linpack, TOP500 e Green500

- O desempenho em Linpack é usado para ranquear os 500 supercomputadores mais rápidos no TOP500.
- O primeiro colocado costuma ser divulgado como o “computador mais rápido do mundo”.
- O benchmark permite reescrever o programa em diferentes linguagens e formas, desde que o resultado e o número de operações sejam preservados.
- O Green500 ordena sistemas por desempenho por Watt ao executar Linpack.

# Modelo de Desempenho e Benchmarks para SMP

## SPECrate

- SPECrate é uma métrica de throughput baseada nos benchmarks SPEC CPU.
- Em vez de medir um único programa isolado, executa várias cópias do programa simultaneamente.
- O objetivo é avaliar paralelismo em nível de tarefa, sem comunicação entre as tarefas.
- O número de cópias pode ser ajustado conforme o sistema testado.
- Portanto, SPECrate também representa uma forma de **weak scaling**.



# Modelo de Desempenho e Benchmarks para SMP

## SPLASH e SPLASH-2

- SPLASH significa *Stanford Parallel Applications for Shared Memory*.
- Foi desenvolvido para avaliar programas paralelos em sistemas de memória compartilhada.
- Inclui kernels e aplicações, muitas delas vindas da computação de alto desempenho.
- Diferentemente de Linpack e SPECrates, exige **strong scaling**.
- SPLASH é mais adequado para estudar sincronização, comunicação e paralelismo em memória compartilhada.

# Modelo de Desempenho e Benchmarks para SMP

## NAS Parallel Benchmarks

- NAS foi criado pela *NASA Advanced Supercomputing* para avaliar multiprocessadores.
- Os benchmarks são inspirados em dinâmica dos fluidos computacional.
- A suíte contém cinco kernels representativos de computação científica.
- Permite **weak scaling** por meio de diferentes conjuntos de dados.
- Assim como Linpack, pode ser reescrito, mas com restrição às linguagens C ou Fortran.

# Modelo de Desempenho e Benchmarks para SMP

## PARSEC

- PARSEC significa *Princeton Application Repository for Shared Memory Computers*.
- É composto por programas multithread que usam Pthreads e OpenMP.
- Seu foco são domínios computacionais emergentes, não apenas aplicações científicas tradicionais.
- Contém nove aplicações e três kernels.
- Inclui paralelismo de dados, paralelismo em pipeline e paralelismo não estruturado.

# Modelo de Desempenho e Benchmarks para SMP

## YCSB: benchmark para serviços em nuvem

- YCSB significa *Yahoo! Cloud Serving Benchmark*.
- Seu objetivo é comparar o desempenho de serviços de dados em nuvem.
- Ele fornece um framework para testar diferentes sistemas de armazenamento e bancos distribuídos.
- Cassandra e HBase são exemplos de sistemas de bancos de dados distribuídos usados que fazem parte do Hadoop.
- YCSB desloca o foco de FLOPs para latência, throughput e comportamento de serviços de dados.

# Modelo de Desempenho e Benchmarks para SMP

## Limitações dos benchmarks tradicionais

- Restrições rígidas tornam os resultados mais justos, mas podem limitar a inovação.
- Melhorias em algoritmos, estruturas de dados e linguagens podem ser proibidas.
- Isso ocorre porque o benchmark busca isolar o efeito da arquitetura e do compilador.
- O problema é que, durante mudanças profundas no modelo de programação, essa restrição pode ser inadequada.
- Benchmarks fechados favorecem comparabilidade; benchmarks abertos favorecem inovação.

# Modelo de Desempenho e Benchmarks para SMP

## Design patterns de Berkeley

- Pesquisadores de Berkeley propuseram avaliar aplicações por padrões de projeto computacionais.
- Foram identificados 13 padrões relevantes para aplicações futuras.
- Exemplos incluem matrizes esparsas, grades estruturadas, máquinas de estados finitos, MapReduce e travessia de grafos.
- A definição em alto nível permite inovação em algoritmos, linguagens, estruturas de dados e arquiteturas.
- Em vez de fixar uma implementação, fixa-se a tarefa computacional.

# Modelo de Desempenho e Benchmarks para SMP

## MLPerf e a ideia de divisões abertas e fechadas

- MLPerf é uma suíte moderna voltada para aprendizado de máquina.
- Inclui programas, bases de dados e regras de submissão.
- Novas versões aparecem frequentemente para acompanhar a evolução rápida da área.
- A divisão fechada impõe regras rígidas para comparações justas entre sistemas.
- A divisão aberta permite inovação em algoritmos, estruturas de dados e sistemas de programação.

# Modelo de Desempenho e Benchmarks para SMP

## Benchmarks e Modelos de desempenho

- Benchmarks dizem **quanto** desempenho um sistema atinge em determinada carga.
- Modelos de desempenho ajudam a explicar **por que** esse desempenho ocorre.
- Com arquiteturas diversas, como multithreading, SIMD e GPUs, modelos simples tornam-se úteis.
- Um bom modelo não precisa ser perfeito; precisa gerar intuições corretas.
- Modelo de desempenho é uma abstração que relaciona características da aplicação com limites da arquitetura.



# Modelo de Desempenho e Benchmarks para SMP

## Kernels paralelos e desempenho de ponto flutuante

- Para muitos kernels paralelos, especialmente científicos, operações de ponto flutuante dominam o tempo de execução.
- O desempenho máximo em ponto flutuante impõe um limite superior ao desempenho do kernel.
- Em chips multicore, o pico de desempenho é a soma do pico de todos os núcleos.
- Em sistemas com múltiplos chips, multiplica-se o pico por chip pelo número de chips.
- Esse limite computacional é uma das bases do **Roofline Model**.

# Modelo de Desempenho e Benchmarks para SMP

## Demanda sobre o sistema de memória

- O desempenho computacional máximo só é útil se a memória conseguir alimentar as unidades de execução.
- A demanda de largura de banda pode ser estimada dividindo FLOPs/s por FLOPs/byte.

$$\text{Bytes/s} = \text{Floating-Point Operations/s} / \text{Floating-Point Operations/byte}$$

- Essa relação conecta computação, acesso à memória e intensidade aritmética.
- Muitos programas paralelos são limitados não pelo número de núcleos, mas pela largura de banda de memória.

# Modelo de Desempenho e Benchmarks para SMP

## Intensidade aritmética

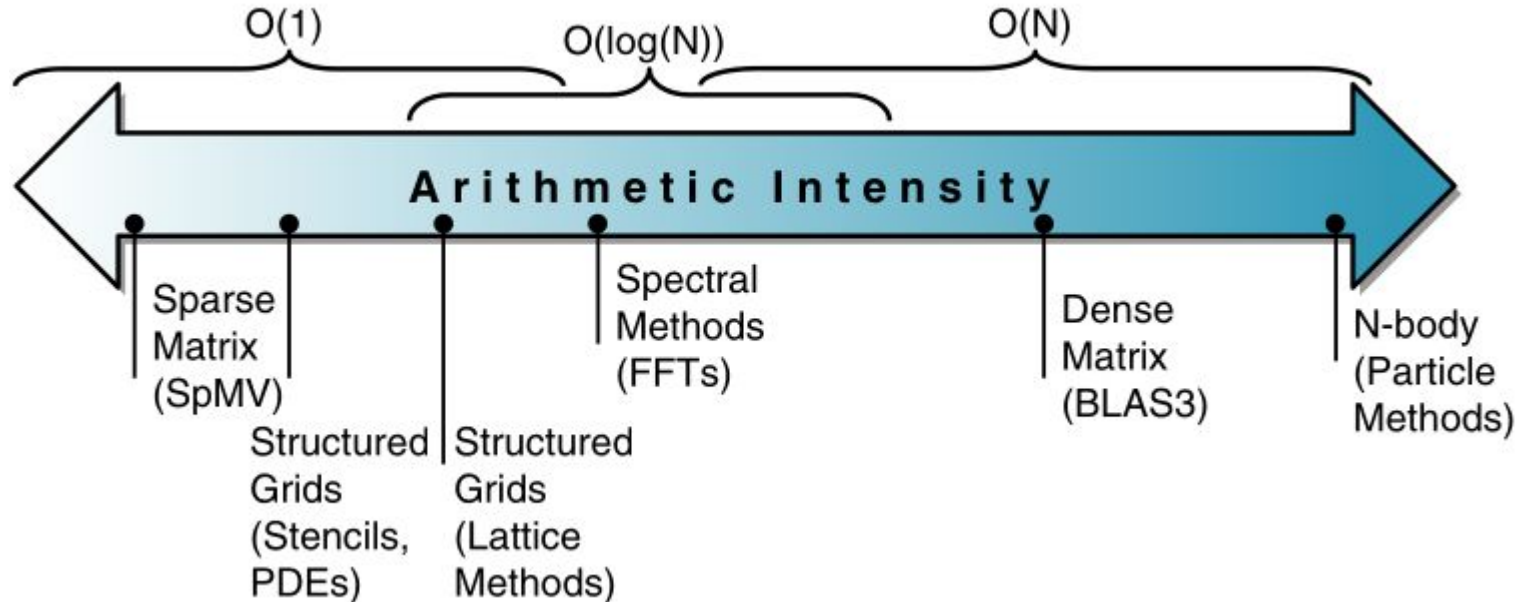
- Intensidade aritmética é a razão entre operações de ponto flutuante e bytes transferidos da memória principal.
- Definição formal:

$$IA = FLOPs \text{ totais} / \text{bytes transferidos da memória principal}$$

- Kernels com baixa intensidade aritmética fazem poucas operações por byte lido ou escrito.
- Kernels com alta intensidade aritmética reutilizam mais dados e realizam mais computação por acesso à memória.
- Intensidade aritmética determina se um kernel tende a ser limitado por memória ou por computação.

# Modelo de Desempenho e Benchmarks para SMP

## Intensidade Aritmética para alguns padrões de projeto do Berkeley



# Modelo de Desempenho e Benchmarks para SMP

## O Roofline Model

- O Roofline Model relaciona desempenho de ponto flutuante, intensidade aritmética e largura de banda de memória.
- Ele representa limites superiores de desempenho em um gráfico bidimensional.
- O eixo Y mostra desempenho alcançável em FLOPs/s.
- O eixo X mostra intensidade aritmética em FLOPs/byte.
- Roofline define um “teto” de desempenho para cada intensidade aritmética.

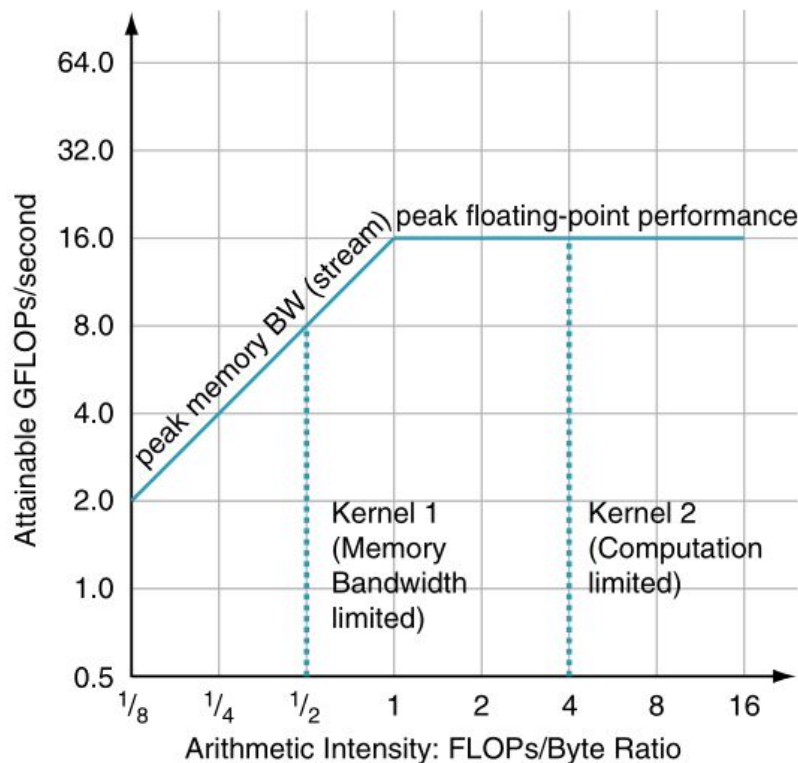
# Modelo de Desempenho e Benchmarks para SMP

## Construção do Roofline Model

- O pico de desempenho computacional é obtido a partir das especificações do hardware.
- O pico de largura de banda de memória pode ser medido por benchmarks como Stream.
- O modelo é construído uma vez para uma máquina, não separadamente para cada kernel.
- Cada kernel é então posicionado no gráfico de acordo com sua intensidade aritmética.
- A partir disso, estima-se o limite máximo de desempenho possível para aquele kernel.

# Modelo de Desempenho e Benchmarks para SMP

## Gráfico Roofline



# Modelo de Desempenho e Benchmarks para SMP

## Fórmula central do Roofline

- O desempenho máximo alcançável é limitado por computação e por memória.
- A parte da memória depende da largura de banda multiplicada pela intensidade aritmética.
- Fórmula:

$$GFLOPs/s = \min(Peak\ Memory\ BW \times Arithmetic\ Intensity, Peak\ Floating-Point\ Performance)$$

- Se o termo da memória for menor, o kernel é limitado por largura de banda.
- Se o pico computacional for menor, o kernel é limitado por computação.



# Modelo de Desempenho e Benchmarks para SMP

## Interpretação geométrica do Roofline

- A linha diagonal representa a limitação de memória.
- A linha horizontal representa a limitação computacional.
- Um kernel com baixa intensidade aritmética intercepta a parte inclinada do teto.
- Um kernel com alta intensidade aritmética intercepta a parte plana do teto.
- A posição no eixo X indica o tipo provável de gargalo.

# Modelo de Desempenho e Benchmarks para SMP

## Kernel limitado por memória

- Um kernel é limitado por memória quando sua intensidade aritmética é baixa.
- Nesse caso, aumentar o número de unidades de ponto flutuante pode ter pouco efeito.
- O desempenho máximo cresce com a largura de banda de memória.
- Otimizações devem reduzir tráfego de memória ou melhorar acesso aos dados.
- Exemplos comuns incluem códigos com baixa reutilização, acesso irregular ou estruturas esparsas.

# Modelo de Desempenho e Benchmarks para SMP

## Kernel limitado por computação

- Um kernel é limitado por computação quando sua intensidade aritmética é alta.
- Nesse caso, a memória consegue fornecer dados em ritmo suficiente.
- O desempenho passa a depender do pico de FLOPs da arquitetura.
- Otimizações devem explorar ILP, SIMD.
- Exemplos típicos incluem multiplicação densa de matrizes e alguns kernels N-body.

# Modelo de Desempenho e Benchmarks para SMP

## Ridge Point (Ponto de Crista)

- O ridge point é o ponto onde a linha diagonal encontra a linha horizontal.
- Ele indica a intensidade aritmética mínima necessária para atingir o pico computacional.
- Se o ridge point está muito à direita, poucos kernels conseguem atingir o pico da máquina.
- Se está muito à esquerda, muitos kernels têm potencial para atingir alto desempenho.
- O ridge point resume o equilíbrio entre poder computacional e largura de banda de memória.

# Modelo de Desempenho e Benchmarks para SMP

## Comparação entre Opteron X2 e Opteron X4

- O Opteron X4 possui quatro núcleos, enquanto o Opteron X2 possui dois.
- Ambos usam o mesmo socket e, portanto, possuem canais de DRAM semelhantes.
- O Opteron X4 também dobra o desempenho de ponto flutuante por núcleo.
- Como as frequências são semelhantes, o Opteron X4 tem cerca de quatro vezes o pico de ponto flutuante.
- Porém, a largura de banda de memória permanece aproximadamente igual.

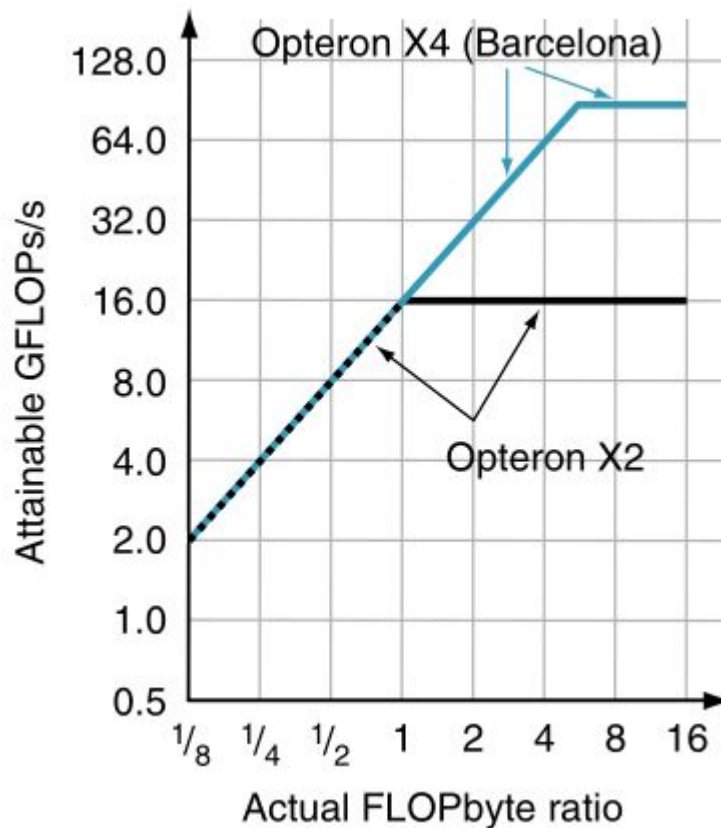
# Modelo de Desempenho e Benchmarks para SMP

## Consequência arquitetural da comparação

- A nova geração aumenta muito o pico computacional, mas não aumenta proporcionalmente a largura de banda de memória.
- Isso desloca o ridge point para a direita.
- Kernels precisam de intensidade aritmética maior para se beneficiar plenamente do novo processador.
- Códigos limitados por memória podem ter pouco ganho, apesar do aumento de núcleos e FLOPs.
- Mais FLOPs não garantem mais desempenho se a memória continuar sendo o gargalo.

# Modelo de Desempenho e Benchmarks para SMP

## Roofline dos dois Opterons



# Modelo de Desempenho e Benchmarks para SMP

## Quando o programa fica abaixo do teto

- O Roofline fornece um limite superior, mas programas reais podem ficar muito abaixo dele.
- A distância até o teto indica oportunidade de otimização.
- A pergunta passa a ser: qual gargalo deve ser atacado primeiro?
- O modelo ajuda a priorizar otimizações computacionais ou de memória.
- Estar abaixo do Roofline não indica apenas baixo desempenho; indica espaço potencial para otimização.



# Modelo de Desempenho e Benchmarks para SMP

## Otimização computacional 1: mistura de operações de ponto flutuante

- O pico computacional muitas vezes exige equilíbrio entre adições e multiplicações.
- Arquiteturas com **FMA** se beneficiam quando o código combina multiplicação e soma.
- Também é necessário que uma fração significativa das instruções seja de ponto flutuante.
- Código com muitas instruções inteiras, controle ou endereçamento pode não atingir o pico.
- O pico teórico assume uma mistura ideal de instruções que programas reais nem sempre possuem.

# Modelo de Desempenho e Benchmarks para SMP

## Otimização computacional 2: ILP e SIMD

- Arquiteturas modernas atingem melhor desempenho quando várias instruções são buscadas, executadas e finalizadas por ciclo.
- O compilador pode aumentar ILP por reordenação e unrolling de loops.
- SIMD permite que uma única instrução opere sobre múltiplos dados.
- Em arquiteturas x86 modernas, instruções AVX podem operar sobre vários operandos de dupla precisão.
- Explorar paralelismo dentro do núcleo é essencial antes de esperar desempenho próximo ao pico.

# Modelo de Desempenho e Benchmarks para SMP

## Otimização de memória 1: software prefetching

- Software prefetching tenta trazer dados para a hierarquia de memória antes de serem usados.
- Isso permite manter várias operações de memória em andamento simultaneamente.
- O objetivo é esconder latência de memória por antecipação.
- Funciona melhor quando os padrões de acesso são previsíveis.
- Prefetching melhora desempenho quando o gargalo é latência ou baixa concorrência de acessos à memória.

# Modelo de Desempenho e Benchmarks para SMP

## Otimização de memória 2: afinidade de memória

- Processadores modernos frequentemente possuem controladores de memória integrados ao chip.
- Em sistemas com múltiplos chips, parte da memória é local e parte é remota.
- Acesso à memória remota atravessa a interconexão entre chips e tem menor desempenho.
- A afinidade de memória tenta alocar threads e dados no mesmo par processador-memória.
- Em sistemas NUMA, onde o dado está localizado pode ser tão importante quanto o algoritmo usado.

# Modelo de Desempenho e Benchmarks para SMP

## Sequência de decisão usando Roofline

- Passo 1: calcular ou medir a intensidade aritmética do kernel.
- Passo 2: posicionar o kernel no eixo X do Roofline.
- Passo 3: identificar se o limite dominante é memória ou computação.
- Passo 4: comparar o desempenho medido com o teto teórico.
- Passo 5: escolher otimizações de acordo com a região e com a distância até os limites.

# Modelo de Desempenho e Benchmarks para SMP

## Ceilings no Roofline Model

- Ceilings são limites intermediários abaixo do teto máximo.
- Cada ceiling representa uma otimização necessária para atingir um nível superior de desempenho.
- Por exemplo, sem SIMD, o desempenho pode ficar abaixo do teto computacional.
- Sem afinidade de memória, o desempenho pode ficar abaixo do teto de largura de banda.
- Um programa não ultrapassa determinado ceiling sem aplicar a otimização correspondente.

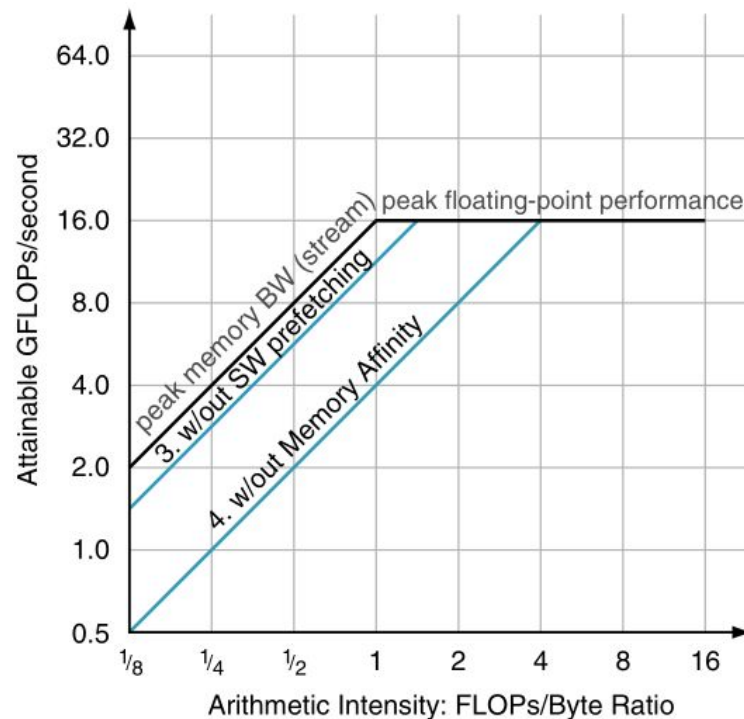
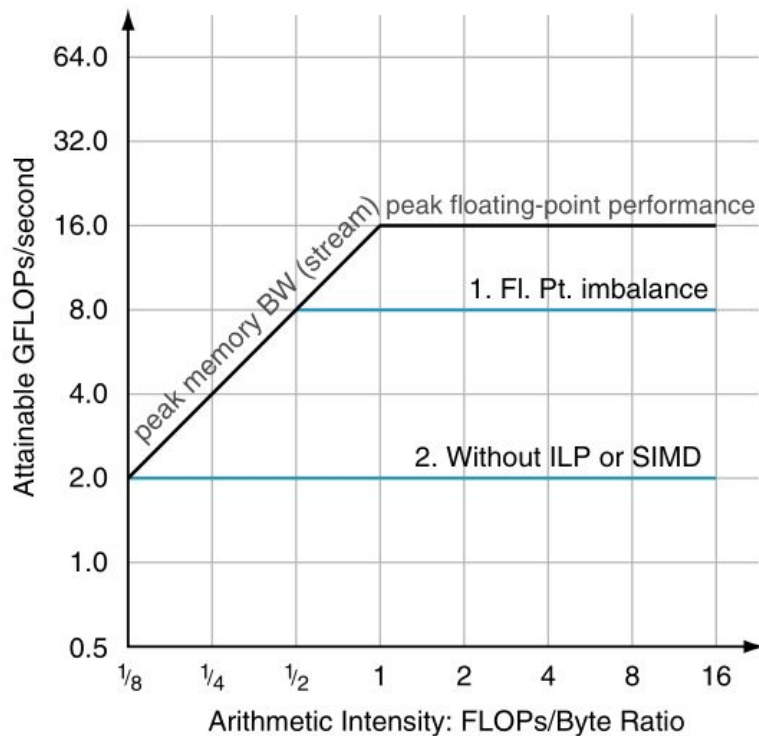
# Modelo de Desempenho e Benchmarks para SMP

## Como os ceilings são determinados?

- O teto computacional pode ser obtido em manuais e especificações da arquitetura.
- O teto de memória pode ser medido com benchmarks como Stream.
- Alguns ceilings computacionais podem ser derivados das capacidades da unidade de ponto flutuante.
- Ceilings de memória, como afinidade, exigem experimentos na máquina específica.
- Uma vez caracterizada a máquina, os resultados podem orientar vários programadores.

# Modelo de Desempenho e Benchmarks para SMP

## Ceilings computacionais e de memória





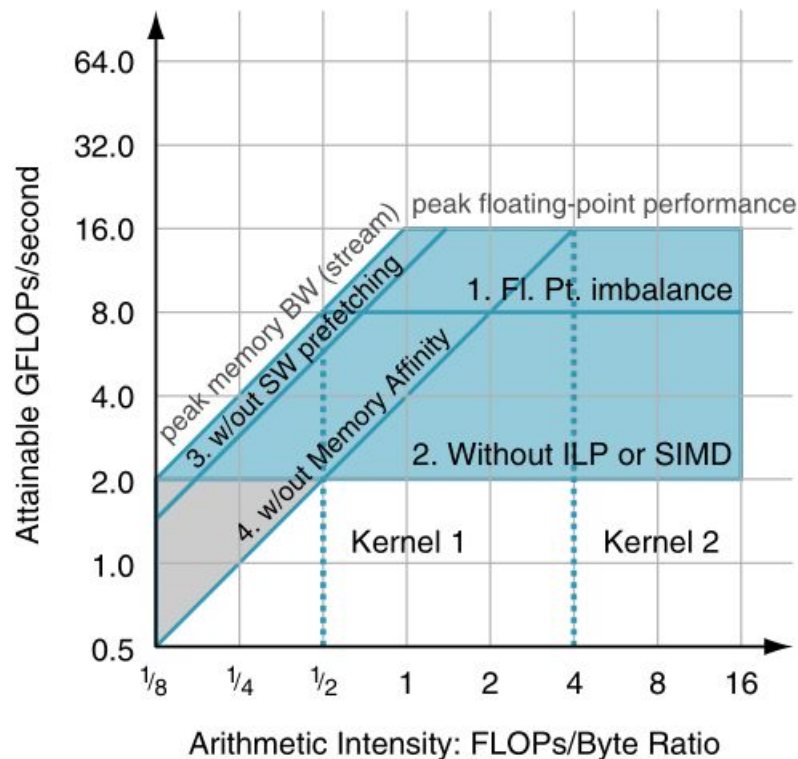
# Modelo de Desempenho e Benchmarks para SMP

## Interpretando recompensas de otimização

- A largura da lacuna entre dois ceilings representa a oportunidade de ganho.
- Se a lacuna associada ao ILP é grande, vale priorizar unrolling, reordenação e vetorização.
- Se a lacuna associada à afinidade de memória é grande, vale priorizar alocação NUMA-aware.
- O modelo evita aplicar otimizações aleatórias sem entender o gargalo dominante.
- Roofline transforma otimização em um processo orientado por limites quantitativos.

# Modelo de Desempenho e Benchmarks para SMP

## Regiões de otimização



# Modelo de Desempenho e Benchmarks para SMP

## Estratégias por região do Roofline

- Região de baixa intensidade aritmética: priorizar redução de tráfego de memória, prefetching e afinidade.
- Região intermediária: combinar otimizações de memória e computação.
- Região de alta intensidade aritmética: priorizar ILP, SIMD, FMA e mistura adequada de operações.
- Um kernel pode mudar de região se sua intensidade aritmética for alterada.
- A melhor otimização depende da posição do kernel no gráfico, não apenas da arquitetura.

# Modelo de Desempenho e Benchmarks para SMP

## Intensidade aritmética não é fixa

- Alguns kernels aumentam a intensidade aritmética conforme o tamanho do problema cresce.
- Isso ocorre em problemas como matrizes densas e N-body.
- Esse comportamento ajuda a explicar por que weak scaling pode ser mais favorável que strong scaling.
- Otimizações de cache também podem aumentar a intensidade aritmética efetiva.
- Portanto, a posição do kernel no Roofline pode mudar após otimizações.

# Modelo de Desempenho e Benchmarks para SMP

## Melhorando intensidade aritmética

- Melhorar localidade temporal reduz acessos repetidos à memória principal.
- Loop unrolling pode permitir reorganizar operações que acessam endereços semelhantes.
- Algumas arquiteturas possuem instruções especiais de cache para evitar leituras desnecessárias.
- Ao reduzir tráfego de memória, o kernel realiza mais FLOPs por byte transferido.
- Aumentar intensidade aritmética desloca o kernel para a direita no Roofline.

# Modelo de Desempenho e Benchmarks para SMP

## Exemplo conceitual: deslocamento no Roofline

- Um kernel inicialmente limitado por memória pode melhorar ao reduzir acessos à DRAM.
- Se a intensidade aritmética aumenta, sua linha vertical desloca-se para a direita.
- Esse deslocamento pode levá-lo de uma região de memória para uma região intermediária.
- Com isso, novas otimizações computacionais passam a fazer sentido.
- O processo de otimização pode ser iterativo: memória primeiro, depois computação.

# Modelo de Desempenho e Benchmarks para SMP

## Uso do Roofline por arquitetos

- O Roofline não serve apenas para programadores otimizarem código.
- Arquitetos podem usá-lo para decidir onde investir recursos de hardware.
- Se kernels importantes são limitados por memória, aumentar FLOPs pode trazer pouco benefício.
- Se kernels têm alta intensidade aritmética, unidades vetoriais e FMA podem ser mais valiosas.
- O modelo ajuda a alinhar projeto de hardware com cargas de trabalho reais.

# Modelo de Desempenho e Benchmarks para SMP

## Síntese

- Benchmarks são essenciais para comparar sistemas multiprocessados, mas exigem regras claras.
- Linpack, SPECrates, SPLASH, NAS, PARSEC, YCSB e MLPerf avaliam aspectos diferentes do desempenho.
- Alguns benchmarks priorizam comparabilidade; outros permitem maior inovação.
- Strong scaling e weak scaling medem propriedades diferentes de sistemas paralelos.
- A escolha do benchmark influencia diretamente a conclusão sobre qual sistema é “melhor”.



# Modelo de Desempenho e Benchmarks para SMP

## Síntese

- Modelos de desempenho explicam limites e gargalos que benchmarks isolados não revelam.
- O Roofline Model relaciona FLOPs/s, largura de banda de memória e intensidade aritmética.
- Kernels podem ser limitados por memória ou por computação, dependendo de sua posição no gráfico.
- Ceilings ajudam a priorizar otimizações específicas.
- O Roofline é uma ferramenta didática e prática para entender desempenho em arquiteturas paralelas.

# Modelo de Desempenho e Benchmarks para SMP

## Síntese

- Benchmarks respondem: **qual sistema executa melhor determinada carga?**
- Modelos de desempenho respondem: **por que o sistema atinge ou não determinado desempenho?**
- Em multiprocessadores modernos, desempenho depende da interação entre paralelismo, memória, SIMD, ILP e software.
- O Roofline Model fornece uma visão integrada desses fatores.

# Bibliografia

Capítulo 6 - Computer Organization and Design RISC-V Edition: The Hardware Software Interface, David A. Patterson, John L. Hennessy. Editora Morgan Kaufmann, 2ª Edição, 2020.